

how to extract a subset of a colormap as a new colormap in matplotlib?

Asked 10 years, 11 months ago Modified 2 years ago Viewed 56k times

I would like to use a colormap from matplotlib e.g. CMRmap. But I don't want to use the "black" color at the beginning and the "white" color at the end. I'm interested to plot my data using the in-between colors. I think ppl use it quite often but I was searching over internet and could not manage to find any simple solution. I'll appreciate if someone suggest any solution.

80

`python` `matplotlib` [Edit tags](#)

[Share](#) [Edit](#) [Follow](#) [Close](#) [Flag](#) [Protect](#)

asked Sep 20, 2013 at 21:17

 rana
1,081 2 11 16

1  V useful to avoid black when the colors are used for text backgrounds - good question – [Bonlenfum](#) Oct 24, 2018 at 14:04

7 Answers

Sorted by: [Reset to default](#) Date modified (newest first)

 `cmap = cmr.get_sub_map('viridis', 0.2, 0.8, N=5)`

0

proposed by @1313e must be the most elegant solution. But the new function is `cmr.get_sub_cmap()` , just replace it.

 [Share](#) [Edit](#) [Follow](#) [Flag](#)

edited Sep 6, 2022 at 14:07

 mnikley
1,635 1 10 23

answered Sep 1, 2022 at 3:40

 Ethan Yin
1

Quick Wrapper Function:

1

```
def sub_cmap(cmap, vmin, vmax):  
    return lambda v: cmap(vmin + (vmax - vmin) * v)
```



Usage:

```
cmap = matplotlib.cm.get_cmap('viridis') # Get your favorite cmap  
new_cmap = sub_cmap(cmap, 0.2, 0.9)  
  
# Do plot or something  
# ...
```

[Share](#) [Edit](#) [Follow](#) [Flag](#)

edited Apr 26, 2022 at 9:27

answered Apr 26, 2022 at 9:22

 catwith
1,225 13 19

 This new cmap is not a matplotlib cmap. – [Vladimir Vargas](#) Jun 13, 2022 at 13:15

 In my CMasher package, I provide the `get_sub_cmap()` -function (<https://cmasher.readthedocs.io/user/usage.html#sub-colormaps>), which takes a colormap and a range, and returns a new colormap containing the requested range.

16

So, for example, if you want to take the colors between 20% and 80% of the `viridis` colormap, you can do that with:



```
import cmasher as cmr  
  
cmap = cmr.get_sub_cmap('viridis', 0.2, 0.8)
```

PS: Do not use `jet` (or `CMRmap`), as they are not perceptually uniform sequential. Instead, use the 5 proper colormaps in matplotlib or the colormaps provided by `cmocean` or my `CMasher`.

EDIT: In the latest version of `CMasher`, one can also use this same function to create a discrete/qualitative colormap out of any colormap by supplying the function with the number of segments to take. For example, if you want to create a qualitative colormap of `viridis` in the 20% to 80% range, you can do this with:

```
cmap = cmr.get_sub_map('viridis', 0.2, 0.8, N=5)
```

[Share](#) [Edit](#) [Follow](#) [Flag](#)

edited Mar 30, 2021 at 0:02

answered Jan 12, 2021 at 4:11

 1313e
1,182 10 18

1  CMasher is the way to go. It was not available when this question was asked, but I think nowadays it is the state-of-the-art solution for to OPs problem. – [Christian Herenz](#) Sep 2, 2021 at 20:39

 Slight improvement of visualization from a [previous answer](#). (inspired by [that](#) answer)

1

```
import numpy as np  
import matplotlib as mpl  
import matplotlib.pyplot as plt  
  
def truncate_colormap(cmap, minval=0.0, maxval=1.0, n=100):  
    ...  
  
https://stackoverflow.com/a/18926541
```

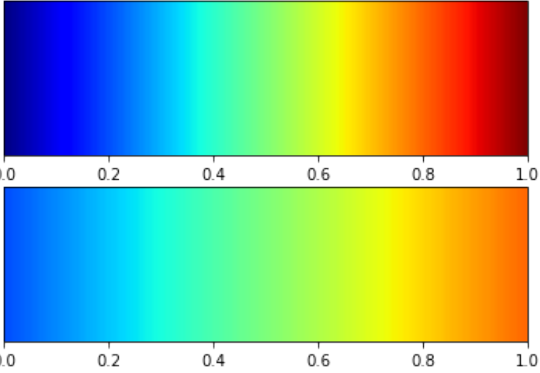
 

```
'''
if isinstance(cmap, str):
    cmap = plt.get_cmap(cmap)
new_cmap = mpl.colors.LinearSegmentedColormap.from_list(
    'trunc({n},{a:.2f},{b:.2f})'.format(n=cmap.name, a=minval, b=maxval),
    cmap(np.linspace(minval, maxval, n)))
return new_cmap

cmap_base = 'jet'
vmin, vmax = 0.2, 0.8
cmap = truncate_colormap(cmap_base, vmin, vmax)

fig, ax = plt.subplots(nrows=2)
sm = mpl.cm.ScalarMappable(cmap=cmap_base)
cbar = plt.colorbar(sm, cax=ax[0], orientation='horizontal')

sm = mpl.cm.ScalarMappable(cmap=cmap)
cbar = plt.colorbar(sm, cax=ax[1], orientation='horizontal')
plt.show()
```



Share Edit Follow Flag

answered Dec 4, 2020 at 11:52

 **icemtel**
472 5 13

Here is an adaptation of a previous answer which embeds the plotting function:

2

```
import matplotlib.pyplot as plt
import matplotlib.colors as colors
import numpy as np

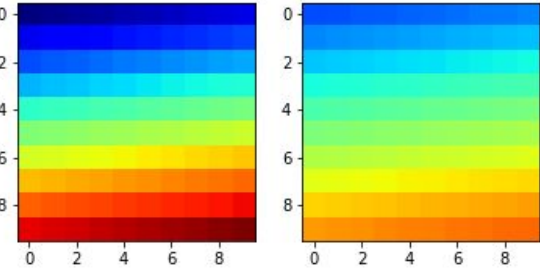
##### Function to truncate color map #####
def truncate_colormap(cmapIn='jet', minval=0.0, maxval=1.0, n=100):
    '''truncate_colormap(cmapIn='jet', minval=0.0, maxval=1.0, n=100)'''
    cmapIn = plt.get_cmap(cmapIn)

    new_cmap = colors.LinearSegmentedColormap.from_list(
        'trunc({n},{a:.2f},{b:.2f})'.format(n=cmapIn.name, a=minval, b=maxval),
        cmapIn(np.linspace(minval, maxval, n)))

    arr = np.linspace(0, 50, 100).reshape((10, 10))
    fig, ax = plt.subplots(ncols=2)
    ax[0].imshow(arr, interpolation='nearest', cmap=cmapIn)
    ax[1].imshow(arr, interpolation='nearest', cmap=new_cmap)
    plt.show()

    return new_cmap

cmap_mod = truncate_colormap(minval=.2, maxval=.8) # calls function to truncate colormap
```



Share Edit Follow Flag

edited Nov 11, 2018 at 21:45

answered Nov 7, 2018 at 14:24

 **fact_finder**
182 2 12

The staticmethod [colors.LinearSegmentedColormap.from_list](#) can be used to create new LinearSegmentedColormaps. Below, I sample the original colormap at 100 points between 0.2 and 0.8:

120

```
cmap(np.linspace(0.2, 0.8, 100))

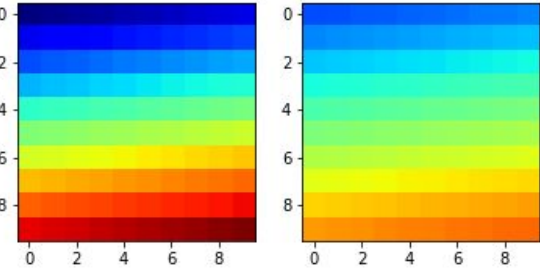
and use these colors to generate a new colormap:

import matplotlib.pyplot as plt
import matplotlib.colors as colors
import numpy as np

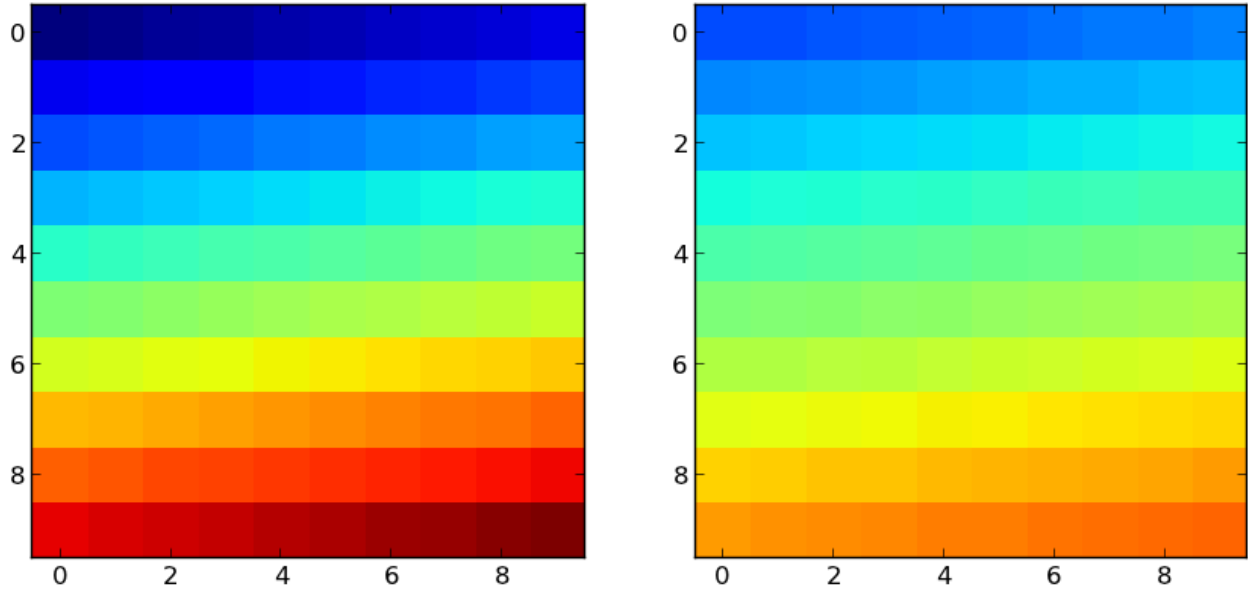
def truncate_colormap(cmap, minval=0.0, maxval=1.0, n=100):
    new_cmap = colors.LinearSegmentedColormap.from_list(
        'trunc({n},{a:.2f},{b:.2f})'.format(n=cmap.name, a=minval, b=maxval),
        cmap(np.linspace(minval, maxval, n)))
    return new_cmap

arr = np.linspace(0, 50, 100).reshape((10, 10))
fig, ax = plt.subplots(ncols=2)

cmap = plt.get_cmap('jet')
new_cmap = truncate_colormap(cmap, 0.2, 0.8)
ax[0].imshow(arr, interpolation='nearest', cmap=cmap)
```



```
ax[1].imshow(arr, interpolation='nearest', cmap=new_cmap)
plt.show()
```



The plot on the left shows the image using the original colormap (in this example, `jet`). The plot on the right shows the same image using `new_cmap`.

Share Edit Follow Flag

edited Sep 21, 2013 at 9:58

answered Sep 20, 2013 at 21:59

 **unutbu**
872k 193 1.8k 1.7k

-  I've taken the liberty of putting this up on [gist.github.com](#) [colormaputil.py](#) together with `get_cmap`, `array_cmap`, `stack_colormap`, `band_colormap`. – **denis** Dec 20, 2013 at 10:15
-  1  Thanks for that. One more question if anyone looks at this again: If I run `truncate_colormap` on a discrete colormap, I get a smooth map in return. How would I return also a discrete map? This is kind of weird, since the function itself returns a linear segmented map – **HansSnah** Sep 4, 2015 at 16:16
-  1  Why `n=100` and not 256? – **Oren** May 15, 2016 at 6:25
-   For those who are very confused like I was by the fact that object `cmap` can accept arguments: class `matplotlib.colors.Colormap` has a method `__call__`, which makes it a functor, i.e. the object can be used as if it is a function. – **biubiuty** Jan 24, 2018 at 21:09
-   For those looking for the [github gist](#) mentioned by @denis – **M.T** Sep 29, 2020 at 9:00

|


11




I was just recently struggling with this on my own. Here are some possible solutions:

Try using `vmin`, `vmax` keyword arguments in your plotting function. For example, say you had data between 0 and 1 but didn't like the colors used at the extremes of the colormap for 0 and 1.

```
import matplotlib.pyplot as plt
import matplotlib.cm as cm

my_cmap = cm.spectral_r
my_cmap.set_over('c')
my_cmap.set_under('m')
plt.pcolor(data, vmin=0.01, vmax=0.99, cmap=my_cmap)
```

This will force the entire colormap to be used for values between 0.01 and 0.99 and values above and below will be cyan and magenta respectively. This may not solve your problem exactly, but it could be useful if you like a particular colormap and wish it had additional colors at both ends.

If you really want to change the colormap, look at the documentation [here](#) and for [LinearSegmentedColormap here](#).

First,

```
import matplotlib.cm as cm
cdict = cm.get_cmap('spectral_r')._segmentdata
```

This returns a dictionary of all the colors that make up the colormap. However, it's pretty tricky figuring out exactly how to alter this dictionary. This dict has three keys, `red`, `green`, `blue`. `cdict[key]` returns a list of values of the form `(x, y0, y1)`. Let's take a look at two consecutive elements of `cdict['red']`:

```
((0.0, 0.0, 0.0)
 (0.5, 1.0, 1.0),...
```

What this means is that data with `z` (assuming we're doing a `pcolor` or `imshow`) between 0.0 and 0.5 will have the red component of the rgb color associated with that data will increase from 0.0 (no red) to 1.0 (maximum red). This means that to change the color of the colormap, you have to examine how each of the three components of rgb are interpolated in the region of the colormap that you are interested in. **Just make sure that for each color, the first and the last entry start with `x=0` and `x=1` respectively; you must cover the whole spectrum of `[0, 1]`.**

If you want to change the beginning and end colors, try

```
import matplotlib.cm as cm
from matplotlib.colors import LinearSegmentedColormap
cdict = cm.get_cmap('spectral_r')._segmentdata
```

```
cdict['red'][0] = (0, 0.5, 0.5) # x=0 for bottom color in colormap
cdict['blue'][0] = (0, 0.5, 0.5) # y=0.5 gray
cdict['green'][0] = (0, 0.5, 0.5) # y1=y for simple interpolation
cdict['red'][-1] = (1, 0.5, 0.5) # x=1 for top color in colormap
cdict['blue'][-1] = (1, 0.5, 0.5)
cdict['green'][-1] = (1, 0.5, 0.5)

my_cmap = LinearSegmentedColormap('name', cdict)
```

Then use this cmap in your plotting function.

What I wanted to do was change the gray at the end of the `spectral_r` colormap to pure white. This was achieved using

```
# Using imports from above
cdict = matplotlib.cm.get_cmap('spectral_r')._segmentdata
cdict['red'][0] = (0, 1, 1)
cdict['green'][0] = (0, 1, 1)
cdict['blue'][0] = (0, 1, 1)
my_cmap = LinearSegmentedColormap('my_cmap', cdict)
```

Share Edit Follow Flag

answered Sep 20, 2013 at 21:43

wflynnny

18.4k64869

 thanks for the informative answer. It will be very useful to construct a custom colormap. – rana Sep 20, 2013 at 22:52